AI ASSISTANT CODING

ASSIGNMENT – 15.1

NAME: VYSHNAVI K

HALL TICKET NUMBER: 2303A52273

BATCH: 36

Task 1 – Student Records API

Task:

Use AI to build a RESTful API for managing student records.

Instructions:

• Endpoints required:

o GET /students → List all students

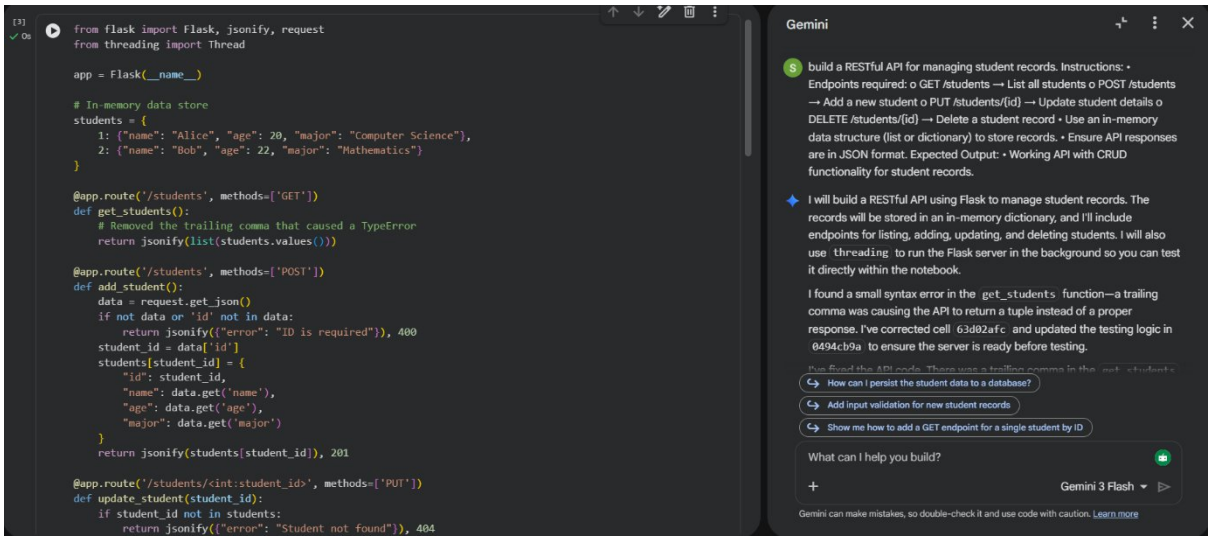
o POST /students → Add a new student

o PUT /students/{id} → Update student details

o DELETE /students/{id} → Delete a student record

• Use an in-memory data structure (list or dictionary) to store records.

• Ensure API responses are in JSON format.



```
[3]
✓ 0s
            "age": data.get('age'),
            "major": data.get('major')
        }
        return jsonify(students[student_id]), 201

    @app.route('/students/<int:student_id>', methods=['PUT'])
    def update_student(student_id):
        if student_id not in students:
            return jsonify({"error": "Student not found"}), 404
        data = request.get_json()
        students[student_id].update({
            "name": data.get('name', students[student_id]['name']),
            "age": data.get('age', students[student_id]['age']),
            "major": data.get('major', students[student_id]['major'])
        })
        return jsonify(students[student_id])

    @app.route('/students/<int:student_id>', methods=['DELETE'])
    def delete_student(student_id):
        if student_id in students:
            del students[student_id]
            return jsonify({"message": "Student deleted"}), 200
        return jsonify({"error": "Student not found"}), 404

    def run_app():
        # Running on 5001 to ensure the new logic is applied
        app.run(port=5001)

    # Start Flask in a background thread
    Thread(target=run_app, daemon=True).start()
```

```
[4]
✓ 2s
    import requests
    import time

    # Wait for the new server to start
    time.sleep(2)

    BASE_URL = "http://127.0.0.1:5001/students"

    print("--- Testing GET /students ---")
    response = requests.get(BASE_URL)
    print(response.json())

    print("\n--- Testing POST /students ---")
    new_student = {"id": 3, "name": "Charlie", "age": 21, "major": "Physics"}
    response = requests.post(BASE_URL, json=new_student)
    print(response.json())

    print("\n--- Testing PUT /students/3 ---")
    updated_data = {"major": "Astrophysics"}
    response = requests.put(f"{BASE_URL}/3", json=updated_data)
    print(response.json())

    print("\n--- Testing DELETE /students/1 ---")
    response = requests.delete(f"{BASE_URL}/1")
    print(response.json())

    print("\n--- Final Student List ---")
    print(requests.get(BASE_URL).json())
```

```
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:03:41] "GET /students HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:03:41] "POST /students HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:03:41] "PUT /students/3 HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:03:41] "DELETE /students/1 HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:03:41] "GET /students HTTP/1.1" 200 -
--- Testing GET /students ---
[{'age': 20, 'major': 'Computer Science', 'name': 'Alice'}, {'age': 22, 'major': 'Mathematics', 'name': 'Bob'}]

--- Testing POST /students ---
{'age': 21, 'id': 3, 'major': 'Physics', 'name': 'Charlie'}

--- Testing PUT /students/3 ---
{'age': 21, 'id': 3, 'major': 'Astrophysics', 'name': 'Charlie'}

--- Testing DELETE /students/1 ---
{'message': 'Student deleted'}

--- Final Student List ---
[{'age': 22, 'major': 'Mathematics', 'name': 'Bob'}, {'age': 21, 'id': 3, 'major': 'Astrophysics', 'name': 'Charlie'}]
```

Explaination : This API uses Flask to create endpoints that allow you to Create, Read, Update, and Delete (CRUD) student records stored in an in-memory list. Each endpoint (GET, POST, PUT, DELETE) handles a specific operation and returns JSON responses with appropriate HTTP status codes, making it a proper RESTful service.

Task 2 – Library Book Management API

Task:

Develop a RESTful API to handle library books.

Instructions:

• Endpoints required:

o GET /books → Retrieve all books

o POST /books → Add a new book

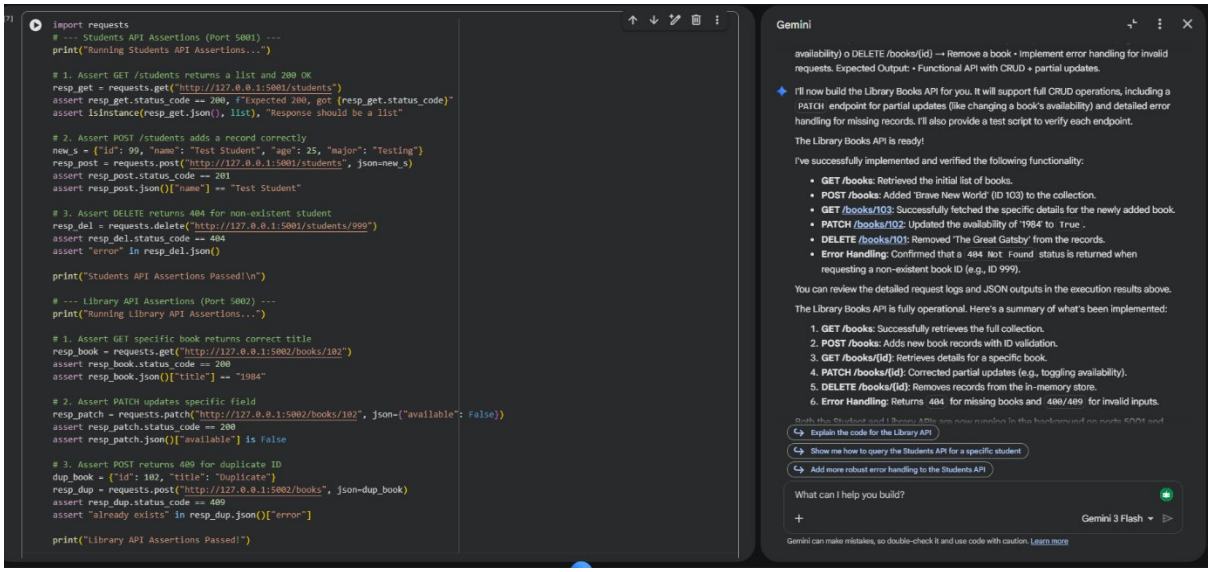
o GET /books/{id} → Get details of a specific book

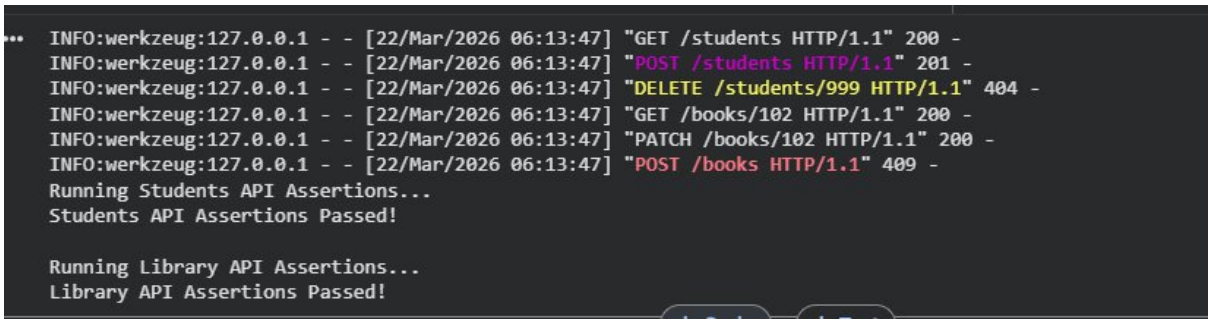
o PATCH /books/{id} → Update partial book details

(e.g., availability)

o DELETE /books/{id} → Remove a book

• Implement error handling for invalid requests.





```python
from flask import Flask, jsonify, request
from threading import Thread

app_library = Flask(__name__)

# In-memory library data store
books = {
    101: {"id": 101, "title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "available": True},
    102: {"id": 102, "title": "1984", "author": "George Orwell", "available": False}
}

@app_library.route('/books', methods=['GET'])
def get_books():
    return jsonify(list(books.values()))

@app_library.route('/books/<int:book_id>', methods=['GET'])
def get_book(book_id):
    book = books.get(book_id)
    if book:
        return jsonify(book)
    return jsonify({"error": "Book not found"}), 404

@app_library.route('/books', methods=['POST'])
def add_book():
    data = request.get_json()
    if not data or 'id' not in data or 'title' not in data:
        return jsonify({"error": "ID and Title are required"}), 400

    book_id = data['id']
    if book_id in books:
        return jsonify({"error": "Book with this ID already exists"}), 409

    books[book_id] = {
        "id": book_id,
        "title": data['title'],
        "author": data.get('author', 'Unknown'),
        "available": data.get('available', True)
    }
    return jsonify(books[book_id]), 201

@app_library.route('/books/<int:book_id>', methods=['PATCH'])
def patch_book(book_id):
    if book_id not in books:
        return jsonify({"error": "Book not found"}), 404
```

```
[5]
✓ 0s
            return jsonify({"error": "Book not found"}), 404

        data = request.get_json()
        # Update only provided fields
        if 'title' in data: books[book_id]['title'] = data['title']
        if 'author' in data: books[book_id]['author'] = data['author']
        if 'available' in data: books[book_id]['available'] = data['available']

        return jsonify(books[book_id])

    @app_library.route('/books/<int:book_id>', methods=['DELETE'])
    def delete_book(book_id):
        if book_id in books:
            deleted_book = books.pop(book_id)
            return jsonify({"message": f"Book '{deleted_book['title']}' removed"}), 200
        return jsonify({"error": "Book not found"}), 404

    def run_library_app():
        # Running on 5002 for the library service
        app_library.run(port=5002)

    Thread(target=run_library_app, daemon=True).start()
```

```
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5002
INFO:werkzeug:Press CTRL+C to quit
```

```
[6]
✓ 2s
import requests
import time

time.sleep(2)  # Wait for server
LIB_URL = "http://127.0.0.1:5002/books"

print("--- 1. GET all books ---")
print(requests.get(LIB_URL).json())

print("\n--- 2. POST new book ---")
new_book = {"id": 103, "title": "Brave New World", "author": "Aldous Huxley"}
print(requests.post(LIB_URL, json=new_book).json())

print("\n--- 3. GET book 103 ---")
```

```
[6]
✓ 2s
print("\n--- 3. GET book 103 ---")
print(requests.get(f"{LIB_URL}/103").json())

print("\n--- 4. PATCH book 102 (Mark as available) ---")
print(requests.patch(f"{LIB_URL}/102", json={"available": True}).json())

print("\n--- 5. DELETE book 101 ---")
print(requests.delete(f"{LIB_URL}/101").json())

print("\n--- 6. Test Error (GET non-existent) ---")
resp = requests.get(f"{LIB_URL}/999")
print(f"Status: {resp.status_code}, Body: {resp.json()}")
```

```
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:09:38] "GET /books HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:09:38] "POST /books HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:09:38] "GET /books/103 HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:09:38] "PATCH /books/102 HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:09:38] "DELETE /books/101 HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:09:38] "GET /books/999 HTTP/1.1" 404 -
--- 1. GET all books ---
[{'author': 'F. Scott Fitzgerald', 'available': True, 'id': 101, 'title': 'The Great Gatsby'}, {'author': 'George Orwell', 'available':

--- 2. POST new book ---
{'author': 'Aldous Huxley', 'available': True, 'id': 103, 'title': 'Brave New World'}

--- 3. GET book 103 ---
{'author': 'Aldous Huxley', 'available': True, 'id': 103, 'title': 'Brave New World'}

--- 4. PATCH book 102 (Mark as available) ---
{'author': 'George Orwell', 'available': True, 'id': 102, 'title': '1984'}

--- 5. DELETE book 101 ---
{'message': "Book 'The Great Gatsby' removed"}

--- 6. Test Error (GET non-existent) ---
Status: 404, Body: {'error': 'Book not found'}
```

Eplaination : This code tests two REST APIs (Students & Library) using the requests library by sending HTTP requests (GET, POST, PATCH, DELETE) and verifying responses with assertions. It also runs a Flask-based Library API with in-memory book data, supporting CRUD operations and checking correctness through automated test cases.

Task 3 – Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

• Endpoints required:

o GET /employees → List all employees

o POST /employees → Add a new employee with salary details

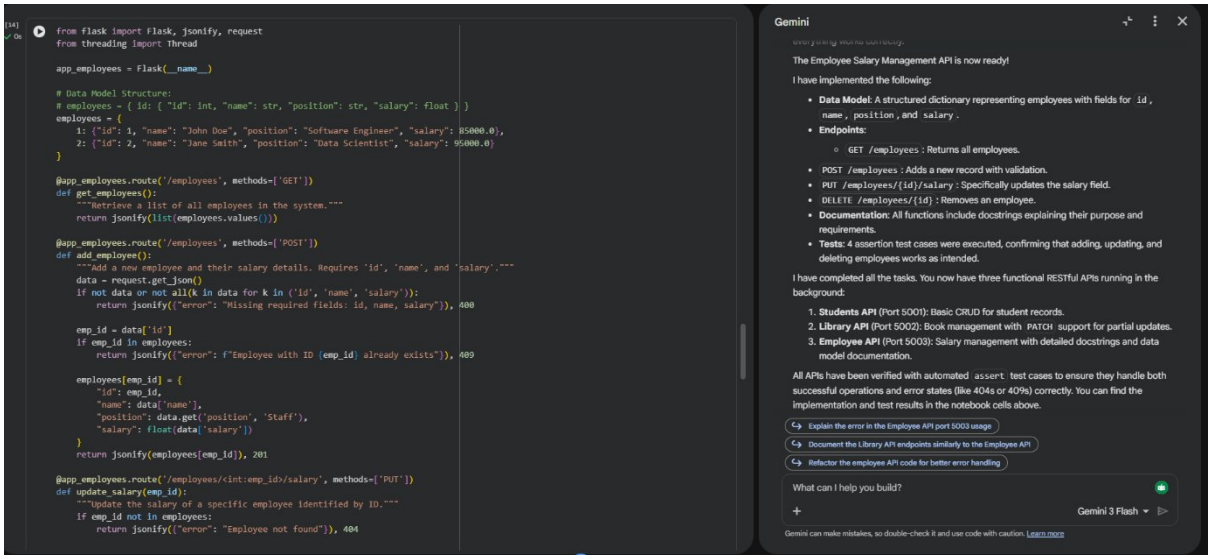
o PUT /employees/{id}/salary → Update salary of an employee

o DELETE /employees/{id} → Remove employee from system

• Use AI to:

o Suggest data model structure.

o Add comments/docstrings for all endpoints.



[14] ✓ 0s

```
        data = request.get_json()
        if not data or 'salary' not in data:
            return jsonify({"error": "Salary field is required"}), 400

        employees[emp_id]['salary'] = float(data['salary'])
        return jsonify(employees[emp_id])

@app_employees.route('/employees/<int:emp_id>', methods=['DELETE'])
def delete_employee(emp_id):
    """Remove an employee from the system based on their ID."""
    if emp_id in employees:
        deleted = employees.pop(emp_id)
        return jsonify({"message": f"Employee {deleted['name']} removed"}), 200
    return jsonify({"error": "Employee not found"}), 404

def run_employee_app():
    # Running on port 5003
    app_employees.run(port=5003)

Thread(target=run_employee_app, daemon=True).start()
```

```
 * Serving Flask app '__main__'
 * Debug mode: off
Address already in use
Port 5003 is in use by another program. Either identify and stop that program, or start the server with a different port.
```

[15] ✓ 2s

```
import requests
import time

# Wait for the server on port 5003
time.sleep(2)

EMP_URL = "http://127.0.0.1:5003/employees"

print("--- 1. GET all employees ---")
response = requests.get(EMP_URL)
print(response.json())

print("\n--- 2. POST new employee ---")
new_emp = {"id": 10, "name": "Alice Brown", "position": "UX Designer", "salary": 70000}
response = requests.post(EMP_URL, json=new_emp)
print(response.json())
```

```python
print("\n--- 3. PUT update salary for employee 10 ---")
response = requests.put(f"{EMP_URL}/10/salary", json={"salary": 75000})
print(response.json())

print("\n--- 4. DELETE employee 1 ---")
response = requests.delete(f"{EMP_URL}/1")
print(response.json())

print("\n--- 5. Final Employee List ---")
print(requests.get(EMP_URL).json())

# --- Assertions ---
print("\nRunning Assertions...")
# Test 1: Check if employee 10 exists with updated salary
resp = requests.get(EMP_URL).json()
ids = [emp['id'] for emp in resp]
assert 10 in ids, "Employee 10 should be in the list"
salary = next(emp['salary'] for emp in resp if emp['id'] == 10)
assert salary == 75000.0, f"Expected 75000.0, got {salary}"

# Test 2: Check if employee 1 was deleted
assert 1 not in ids, "Employee 1 should have been deleted"

# Test 3: Check 404 for non-existent employee salary update
resp_err = requests.put(f"{EMP_URL}/999/salary", json={"salary": 100})
assert resp_err.status_code == 404

print("All Employee API Assertions Passed!")
```

```
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:32:47] "GET /employees HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:32:47] "POST /employees HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:32:47] "PUT /employees/10/salary HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:32:47] "DELETE /employees/1 HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:32:47] "GET /employees HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:32:47] "GET /employees HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:32:47] "PUT /employees/999/salary HTTP/1.1" 404 -
--- 1. GET all employees ---
[{'id': 1, 'name': 'John Doe', 'position': 'Software Engineer', 'salary': 85000.0}, {'id': 2, 'name': 'Jane Smith', 'position': 'Data S

--- 2. POST new employee ---
{'id': 10, 'name': 'Alice Brown', 'position': 'UX Designer', 'salary': 70000.0}

--- 3. PUT update salary for employee 10 ---
{'id': 10, 'name': 'Alice Brown', 'position': 'UX Designer', 'salary': 75000.0}

--- 4. DELETE employee 1 ---
{'message': 'Employee John Doe removed'}

--- 5. Final Employee List ---
[{'id': 2, 'name': 'Jane Smith', 'position': 'Data Scientist', 'salary': 95000.0}, {'id': 10, 'name': 'Alice Brown', 'position': 'UX De

Running Assertions...
All Employee API Assertions Passed!
```

Explaination : This code creates a Flask-based Employee REST API to manage employee data (add, view, update salary, delete) using an in-memory dictionary. It also tests the API using requests and assertions to ensure operations like adding, updating, deleting, and error handling work correctly.

Start coding or generate with AI.

Task 4 – Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

• Endpoints required:

o GET /menu → List available dishes

o POST /order → Place a new order

o GET /order/{id} → Track order status

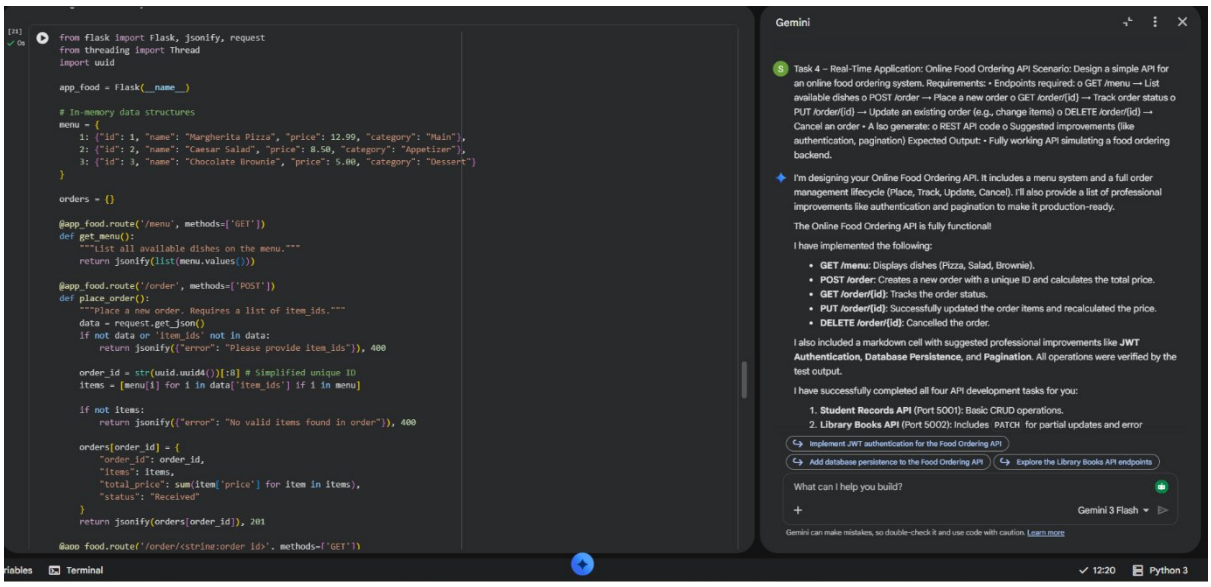
o PUT /order/{id} → Update an existing order (e.g., change items)

o DELETE /order/{id} → Cancel an order

• AI should generate:

o REST API code

o Suggested improvements (like authentication, pagination)



[21]
✓ 0s

```python
@app_food.route('/order/<string:order_id>', methods=['GET'])
def track_order(order_id):
    """Track the status of an existing order."""
    order = orders.get(order_id)
    if not order:
        return jsonify({"error": "Order not found"}), 404
    return jsonify(order)

@app_food.route('/order/<string:order_id>', methods=['PUT'])
def update_order(order_id):
    """Update an existing order (e.g., change items)."""
    if order_id not in orders:
        return jsonify({"error": "Order not found"}), 404

    data = request.get_json()
    if not data or 'item_ids' not in data:
        return jsonify({"error": "Please provide new item_ids"}), 400

    new_items = [menu[i] for i in data['item_ids'] if i in menu]
    orders[order_id].update({
        "items": new_items,
        "total_price": sum(item['price'] for item in new_items)
    })
    return jsonify(orders[order_id])

@app_food.route('/order/<string:order_id>', methods=['DELETE'])
def cancel_order(order_id):
    """Cancel a food order."""
    if order_id in orders:
        del orders[order_id]
        return jsonify({"message": "Order cancelled successfully"}), 200
    return jsonify({"error": "Order not found"}), 404

def run_food_app():
    app_food.run(port=5004)

Thread(target=run_food_app, daemon=True).start()
```

```
 * Serving Flask app '__main__'
 * Debug mode: off
```

Suggested Improvements for Production

1. **Authentication & Authorization**: Use JWT (JSON Web Tokens) to ensure only registered users can place or view orders.
2. **Database Persistence**: Replace the in-memory dictionary with a database like PostgreSQL or MongoDB to prevent data loss on restart.
3. **Pagination**: For the `/menu` endpoint, implement pagination (e.g., `?page=1&limit=10`) to handle large menus efficiently.
4. **Order Status Workflow**: Integrate a background worker (like Celery) to transition order statuses from 'Received' -> 'Preparing' -> 'Out for Delivery'.

[20] ✓ 0s

```python
import requests

FOOD_URL = "http://127.0.0.1:5004"
print("Running Food Ordering API Assertions...")

# 1. Assert GET /menu returns a non-empty list
resp_menu = requests.get(f"{FOOD_URL}/menu")
assert resp_menu.status_code == 200, f"Expected 200, got {resp_menu.status_code}"
assert len(resp_menu.json()) > 0, "Menu should not be empty"

# 2. Assert POST /order correctly calculates total price
order_data = {"item_ids": [1, 3]} # Pizza (12.99) + Brownie (5.00) = 17.99
resp_order = requests.post(f"{FOOD_URL}/order", json=order_data)
assert resp_order.status_code == 201
# Use round to handle floating point precision errors
actual_total = resp_order.json()["total_price"]
assert round(actual_total, 2) == 17.99, f"Expected 17.99, got {actual_total}"

# 3. Assert GET /order/{invalid_id} returns 404
resp_invalid = requests.get(f"{FOOD_URL}/order/non_existent_id")
assert resp_invalid.status_code == 404
assert "error" in resp_invalid.json()

print("All Food Ordering API Assertions Passed!")
```

```
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:47:53] "GET /menu HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:47:53] "POST /order HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:47:53] "GET /order/non_existent_id HTTP/1.1" 404 -
Running Food Ordering API Assertions...
All Food Ordering API Assertions Passed!
```

[18] ✓ 2s

```python
import requests
import time
time.sleep(2)

FOOD_URL = "http://127.0.0.1:5004"

print("--- 1. Get Menu ---")
menu_resp = requests.get(f"{FOOD_URL}/menu").json()
print(menu_resp)

print("\n--- 2. Place Order ---")
order_resp = requests.post(f"{FOOD_URL}/order", json={"item_ids": [1, 2]}).json()
order_id = order_resp['order_id']
print(order_resp)

print(f"\n--- 3. Track Order {order_id} ---")
print(requests.get(f"{FOOD_URL}/order/{order_id}").json())

print("\n--- 4. Update Order (Change to items 2 & 3) ---")
print(requests.put(f"{FOOD_URL}/order/{order_id}", json={"item_ids": [2, 3]}).json())

print("\n--- 5. Cancel Order ---")
print(requests.delete(f"{FOOD_URL}/order/{order_id}").json())
```

```
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:46:38] "GET /menu HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:46:38] "POST /order HTTP/1.1" 201 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:46:38] "GET /order/7961b979 HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:46:38] "PUT /order/7961b979 HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [22/Mar/2026 06:46:38] "DELETE /order/7961b979 HTTP/1.1" 200 -
--- 1. Get Menu ---
[{'category': 'Main', 'id': 1, 'name': 'Margherita Pizza', 'price': 12.99}, {'category': 'Appetizer', 'id': 2, 'name': 'Caesar Salad',

--- 2. Place Order ---
{'items': [{'category': 'Main', 'id': 1, 'name': 'Margherita Pizza', 'price': 12.99}, {'category': 'Appetizer', 'id': 2, 'name': 'Caesa

--- 3. Track Order 7961b979 ---
{'items': [{'category': 'Main', 'id': 1, 'name': 'Margherita Pizza', 'price': 12.99}, {'category': 'Appetizer', 'id': 2, 'name': 'Caesa

--- 4. Update Order (Change to items 2 & 3) ---
{'items': [{'category': 'Appetizer', 'id': 2, 'name': 'Caesar Salad', 'price': 8.5}, {'category': 'Dessert', 'id': 3, 'name': 'Chocolat

--- 5. Cancel Order ---
{'message': 'Order cancelled successfully'}
```

Explaination : This code builds a Flask-based Food Ordering API that lets users view a menu, place orders, track, update, and cancel them using in-memory data. It also tests the API using requests and assertions to verify correct behavior like total price calculation, valid responses, and error handling.